

The Afterlife of Devices

SUBThe Afterlife of Devices

p-rick research program

p-rick — an independent research program on missing software: Vendor-death resilience for connected hardware.

The Afterlife of Devices

SUBThe Afterlife of Devices

p-rick research program

Working Paper P-008 · v1.0 · October 7, 2026 · CC BY 4.0

Abstract — The connected devices in a typical household — locks, lights, thermostats, cameras, hubs, irrigation controllers — have physical lifetimes of a decade or more, and corporate lifetimes of a few years. When the vendor dies, or is acquired, or simply decides to stop, the devices enter a state nobody designed for: they keep existing while their software stops existing. Insteon’s 2023 shutdown darkened hubs, locks, and sensors overnight. Google’s 2016 Revolv shutdown bricked a \$300 hub deliberately. Chamberlain’s 2023 integration lockout degraded myQ openers that had worked locally for years. Each event was called an outrage; none was treated as a *specification failure*. We argue it is exactly that: device ecosystems have no lifecycle model for vendor death, so the outcome is improvised — sometimes rescued by volunteers, sometimes litigated, usually landfilled. We specify **succession-ready connected devices**: a formal lifecycle state machine with an explicit *succession transition*; a **device succession manifest** declaring each device’s local-capability floor and its cloud-dependency graph; an **escrowed continuity protocol** in which firmware, specifications, and signing authority are placed in threshold escrow while the vendor is alive, released by verifiable dead-hand triggers (bankruptcy filings, domain lapse, certificate expiry, renewal-proof silence) rather than by anyone’s goodwill; a **key-succession ceremony** that hands code-signing authority to a community quorum without a trusted center; and a **fossil mode** guarantee that core physical functions survive even when nothing else does. The adjacent fields each hold one slice — source-code escrow (enterprise software practice since the 1980s), right-to-repair law, the Matter interoperability standard, the Home Assistant reverse-engineering culture, degradation contracts (P-004) — and none composes them into operational continuity. The gap is **STRONG**: the failure mode is frequent, catastrophic, and recurring; the mitigation components are proven individually; the composition does not exist anywhere.

Keywords: p-rick

p-rick working paper P-008 · series III (continuity) · draft 1.0

The connected devices in a typical household — locks, lights, thermostats, cameras, hubs, irrigation controllers — have physical lifetimes of a decade or more, and corporate lifetimes of a few years. When the vendor dies, or is acquired, or simply decides to stop, the devices enter a state nobody designed for: they keep existing while their software stops existing. Insteon’s 2023 shutdown darkened hubs, locks, and sensors overnight. Google’s 2016 Revolv shutdown bricked a \$300 hub deliberately. Chamberlain’s 2023 integration lockout degraded myQ openers that had worked locally for years. Each event was called an outrage; none was treated as a *specification failure*. We argue it is exactly that: device ecosystems have no lifecycle model for vendor death, so the outcome is improvised — sometimes rescued by volunteers, sometimes litigated, usually landfilled. We specify **succession-ready connected devices**: a formal lifecycle state machine with an explicit *succession transition*; a **device succession manifest** declaring each device’s local-capability floor and its cloud-dependency graph; an **escrowed continuity protocol** in which firmware, specifications, and signing authority are placed in threshold escrow while the vendor is alive, released by verifiable dead-hand triggers (bankruptcy filings, domain lapse, certificate expiry, renewal-proof silence) rather than by anyone’s goodwill; a **key-succession ceremony** that hands code-signing authority to a community quorum without a trusted center; and a **fossil mode** guarantee that core physical functions survive even when nothing else does. The adjacent fields each hold one slice — source-code escrow (enterprise software practice since the 1980s), right-to-repair law, the Matter interoperability standard, the Home Assistant reverse-engineering culture, degradation contracts (P-004) — and none composes them into operational

continuity. The gap is STRONG: the failure mode is frequent, catastrophic, and recurring; the mitigation components are proven individually; the composition does not exist anywhere.

1 Introduction

A light switch lasts thirty years. A light-bulb company lasts five. This arithmetic is the central, unaddressed fact of the connected-device industry: hardware designed to outlive its maker by an order of magnitude, coupled to a software organization that can vanish in an afternoon.

The industry’s answer to date has been to not have an answer. Devices are shipped *cloud-coupled*: their most basic functions depend on a vendor-operated backend, an authentication service, a signing authority, or a configuration API. The coupling is invisible at purchase time, structural at death time. When Insteon abruptly ceased operations in April 2023, its cloud went dark with no notice; hubs that had coordinated locks, lights, and sensors for years became decorative objects. When Google retired the Revolv home-automation hub in 2016, it did so on a published schedule — and bricked the hardware on that schedule, offering a refund instead of a downgrade. When Chamberlain decided in 2023 to block local API access to myQ garage openers, the devices continued to function, but the *behavior owners had paid for* — integration with their own home systems — died by policy. The pattern repeats across Wink’s collapse, Sonos’s legacy-device disputes, and dozens of smaller vendors whose deaths never made the news because their installed bases were too small to mourn publicly.

Three observations sharpen the problem. First, these events are not breaches of contract in any actionable sense — they *are* the contract, read closely. Second, each event produces an identical aftermath: a volunteer scramble (the Insteon resurrection effort), a partial rescue, a wave of e-waste, and no systemic change. Third, and most tellingly, nobody anywhere has written down what *should* happen. There is no lifecycle state, no protocol, no artifact, no instrument through which a device survives the death of its vendor. The event is as undesigned as an earthquake.

This paper writes it down. We treat vendor death as a **first-class lifecycle transition** that can be specified, contracted, escrowed, and executed — the way enterprise software has treated source-code escrow for four decades, and the way DNS treats delegation failure. The thesis in one sentence: **a device’s useful life should not be a contingent consequence of its vendor’s corporate life, and the mechanism that decouples them can be fully specified today.**

2 The failure taxonomy: five ways a vendor dies

Vendor death is not one event but a family of events with different signatures, and a succession architecture must handle each.

Insolvency. The company runs out of money; servers stop; nobody answers. Insteon (2023) is the canonical case: an abrupt shutdown with no transition plan, followed by a partial community resurrection after the brand was acquired. Insolvency is the hardest case because there is no cooperating counterparty — but it is also the most *legally verifiable* (bankruptcy filings are public records), which matters for the dead-hand triggers of §6.

Strategic sunset. The vendor is alive and simply retires the product line. Google’s Revolv shutdown (2016) — a hub deliberately disabled after Google acquired the underlying team — is the canonical case. The vendor is solvent, communicative, and uninterested. Any succession mechanism must survive a counterparty who is healthy enough to interfere and uninterested in cooperating.

Ecosystem consolidation. A platform retires third-party integrations in favor of its own stack. The 2016 “Works with Nest” shutdown did this at ecosystem scale; the Chamberlain myQ lockout of Home Assistant (2023) did it at device scale. The device survives; its *interoperability* dies.

Acquisition churn. The acquirer retires or repurposes the acquired product line, or imposes account migration that breaks older hardware. Sonos’s long-running legacy-device disputes — including a 2020 trade-in program that permanently disabled traded-in devices (“recycle mode”) — illustrate a vendor using its technical authority over hardware it has already sold.

Neglect death. No announcement; the cloud simply rots. Certificates expire, TLS versions age out, domains lapse, mobile apps fall off the stores. Neglect death is the most common and least reported: for every Insteon, hundreds of vendors quietly stop renewing. It matters because it is *mechanically detectable* without anyone’s cooperation.

From the device’s perspective these five modes differ in one property that matters: **whether a counterparty exists who could deliberately interfere with succession.** Insolvency and neglect leave no active adversary. Sunset, consolidation, and acquisition leave a healthy one. The architecture of §6 is designed around exactly this distinction.

3 The landscape: every slice exists, the center is empty

A claim of a missing category must survive a survey. Each adjacent field holds one component of a solution; none composes them.

Interoperability standards (Matter, Thread, Zigbee, Z-Wave). Matter in particular is frequently proposed as the answer to device longevity. It is not, and the distinction is load-bearing: Matter specifies how *cooperating live devices* talk to each other. It specifies nothing about what happens when a vendor’s commissioning backend, account service, or firmware-update authority disappears. A Matter device can still lose its cloud-dependent features, its updates, and its security posture on vendor death. Interoperability is a *protocol* property; succession is a *lifecycle* property. The industry has built the first and assumed the second.

Enterprise source-code escrow. Since the 1980s, enterprise software contracts routinely place source code with an escrow agent, released to the licensee on defined trigger events (vendor insolvency, support failure). This is the strongest existence proof that *escrowed continuity is contractually routine* — for million-dollar deals between sophisticated parties. It has never been productized for a \$60 lightbulb. The consumer setting differs in three ways worth specifying: licensees are millions of strangers with no contract privity; the asset is firmware plus signing authority, not just source; and the trigger must be verifiable without litigation. Escrow industry practice is the precedent the paper scales down.

Right-to-repair law and policy. A real and accelerating movement: warranty-void restrictions (US FTC enforcement, 2021, and the 2024 Massachusetts automotive-right-to-repair law), the French reparability index (mandatory since 2021, with a durability index legislated to follow), the EU ecodesign regulations mandating spare-parts and update availability windows, and US state laws covering electronics. But repair law governs *physical* restoration and parts. Firmware release on vendor death — the exact thing a succession protocol needs — is governed by copyright, licensing, and the DMCA’s §1201 anti-circumvention regime, where the Library of Congress’s triennial rulemaking has granted only narrow class exemptions (repair and security research in specific categories). Repair law fixes the hand; succession fixes the mind.

The reverse-engineering culture. Home Assistant, ESPHome, Tasmota, and their communities represent the world’s most impressive *unsolicited* succession system: thousands of volunteers reverse-engineer dead and dying devices, write local replacements, and re-host functionality. The Insteon partial resurrection of 2023 was community work. Two structural limits: it is illegal-adjacent (circumvention), unreliable (depends on a device being popular enough to attract a maintainer — the long tail of devices dies unmourned), and *unsigned*: community firmware on a locked bootloader is impossible; on an unlocked one it is a security downgrade. The culture proves demand and capability; it cannot be the mechanism, because it lacks standing, keys, and coverage.

Degradation contracts (P-004). This program’s earlier paper specifies how software *behaves under resource pressure* — an ordered, contractual ladder of reduced fidelity. Vendor death is a different resource axis with the same shape: an input the system depended on disappears, and the correct response is a *specified* degradation, not an improvised one. The succession ladder of §5 is a degradation contract over the vendor-dependency axis; the papers share the model deliberately, and §5 states exactly where the analogy holds and breaks.

Dependable-systems research. Literature on graceful degradation, fail-operational systems, and long-life embedded systems supplies vocabulary (fail-operational, safe-state) but assumes the *system* is the artifact whose internal components fail — not that the artifact’s *manufacturer* fails while the artifact is healthy. The economics of the problem (shrouded attributes, adverse selection) has clean treatments (Gabaix & Laibson’s shrouded-attributes model, 2006) but no engineering counterpart.

The center — a pre-agreed, escrowed, key-carrying, verifiable mechanism by which a device outlives its vendor — is empty. VERDICT: the gap is **STRONG**: recurring catastrophic events, no incumbent of any size, every component individually proven, and a decade of volunteer demand demonstrated by the reverse-engineering culture.

4 Why the gap persists

Liability asymmetry. A vendor that plans for its own death writes evidence for plaintiffs in the cases where it degrades service while alive. No GC wants a document titled “what we owe you when we stop existing.”

Shrouded attributes. Buyers cannot observe longevity terms at purchase, so the market cannot price them (Gabaix & Laibson’s model applies precisely: shrouded attributes are over-produced by competitive sellers). Cloud coupling that saves the vendor per-unit cost is thus structurally over-supplied and survives every outage scandal, because the scandal happens after the sale.

The coordination window closes at death. Community takeover requires firmware, specifications, and signing authority — things only the *live* vendor can provide, and precisely the things it has least incentive to provide the closer it gets to being dead. The protocol must be armed while everyone is happy, which is why escrow-at-sale (§6) rather than request-at-death is the load-bearing design decision.

The key-continuity problem looks unsolved. Firmware signing keys in the hands of a dead company rot; keys handed to a community quorum look like a malware pipeline. It *is* solvable — §6.3 gives the ceremony — but the folklore that “you can’t just give away signing keys” has discouraged attempts.

Boot chains assume the vendor forever. Modern devices burn vendor public keys into ROM for secure boot. Well-intentioned security engineering *manufactured* the bricking problem: the better the secure boot, the harder the rescue. Succession-ready devices must treat key succession as a design requirement of the boot chain itself.

5 The model: lifecycle, ladder, and floor

5.1 Lifecycle state machine

A cloud-coupled device occupies a defined lifecycle, and for the first time the post-vendor states are *specified* rather than improvised:

- **LIVE** — vendor operates the cloud, ships updates, honors the manifest (§5.2).
- **ANNOUNCED** — vendor publishes an end-of-support date; manifest’s local floor (§5.3) activates automatically.
- **SUCCESSION** — dead-hand trigger fired (§6.2); escrow opens; community quorum forms.
- **COMMUNITY** — community-signed firmware, local or community-hosted services, per the manifest.
- **FOSSIL** — no software maintenance at all; physical floor functions guaranteed by hardware design (§5.4).

The two novel transitions are LIVE→ANNOUNCED (which must be *automatic*, not negotiated at death time) and SUCCESSION→COMMUNITY (which must carry signing authority without a trusted center). Everything else in the architecture serves those two transitions.

5.2 The device succession manifest (DSM)

Each product line ships a signed, machine-readable manifest — the contract that makes longevity observable at purchase time:

1. **Cloud-dependency graph.** Every runtime capability, marked `local`, `vendor-cloud`, or `community-portable`, with the services each cloud capability requires. This is the artifact reviewers, procurement, and regulators read.
2. **Escrow declaration.** Firmware source (or verified buildable artifacts), board schematics, protocol specifications, and provisioning secrets schedule — the release set, committed at escrow (§6.1).
3. **Succession policy.** Dormancy thresholds, quorum definition, accepted community-operator formats (e.g., a specified Home Assistant/ESPHome target profile), and the choice of fossil floor.
4. **Key-succession plan.** The ceremony parameters of §6.3, binding while the vendor is alive.

The DSM is to device longevity what a nutrition label is to food: not a guarantee of health, but the end of invisibility. The French reparability index demonstrates that such labels are already legislatable.

5.3 The succession ladder

When the ANNOUNCED or SUCCESSION state is entered, capability degrades along a *declared* ladder — a degradation contract (P-004) over the vendor-dependency axis:

- **Tier 0 (full):** all cloud features live.
- **Tier 1 (local core):** the manifest’s local floor — everything marked `local` continues; cloud features queue or degrade to manual control.
- **Tier 2 (community core):** community firmware provides the `community-portable` set at parity or reduced fidelity, per a published changelog.
- **Tier 3 (fossil):** hardware-guaranteed floor (§5.4).

The ladder’s safety requirement: **downward transitions are total but recoverable** — a device in Tier 2 must re-enter Tier 1 automatically if the vendor revives (this happens; Insteon partially did). Hysteresis is inherited from the P-004 model: no thrash between tiers on flapping renewal signals (§6.2 requires sustained silence, not a single missed ping).

5.4 Fossil mode: the hardware floor

Some classes of devices can kill people or strand them: locks, garage openers, thermostats in winter, medical-adjacent sensors. For these, the manifest certifies a **fossil floor** — a set of functions guaranteed by *hardware alone*: a manual deadbolt, a physical garage button, a dumb-thermostat mode, a local alarm independent of any service. Fossil mode is a manufacturing requirement (a physical path that no firmware state can disable), and it is the paper’s answer to the strongest objection (“software solutions can’t guarantee anything about vendor death”): at the bottom of the ladder sits physics, not policy.

6 The protocol

6.1 Escrow while alive

At first general availability, the vendor deposits the DSM’s release set with **N independent escrow agents** (a mature industry; agents hold sealed release sets and are paid whether or not release ever occurs). The deposit is *renewed periodically*: each renewal is a signed proof published to a public append-only log. Renewal is deliberately cheap — a signed timestamp — and deliberately public.

6.2 Dead-hand release

Release requires a **trigger** and a **quorum**. Triggers are events verifiable from public records, without any escrow agent’s discretion:

- bankruptcy or receivership filing by the vendor (public court records);
- domain non-renewal beyond a threshold (WHOIS/RDAP-observable);
- TLS certificate non-renewal on the manifest’s declared service endpoints beyond a threshold;
- silence: no published renewal proof for θ days (θ chosen per device class; e.g., 90 days for light-duty, 180 with amber warnings at 60).

False positives are the design constraint: renewal proofs are public *and* mirrorable, so silence is hard to fake unintentionally, and the ladder’s hysteresis means an accidental release is recoverable — the vendor renews, devices re-enter Tier 1. The 2023 Insteon resurrection (brand acquired, service partially restored months later) is the exact scenario the recoverable-transition requirement is drawn from.

On trigger, k-of-N escrow agents open the release set to the public, and the community-formation protocol begins.

6.3 The key-succession ceremony

The hard part: devices must *accept* community firmware, and accept it only from a legitimate successor, after the vendor’s signing keys are orphaned.

1. **At manufacture**, the device is provisioned with the vendor root key *and* a **continuity commitment**: a hash of a future “succession root” — not the key itself (it does not exist yet), but a specification of the quorum that will define it: a threshold t of n pre-committed identities (escrow agents plus independent stewards).
2. **On succession**, the t -of- n quorum generates the succession root and signs the *first* community release. The device bootloader verifies the quorum signature against the burned-in commitment — a decentralization of trust performed while everyone was alive, exercised only when they are not.
3. **Thereafter**, the succession root rotates on the ordinary rules; every device knows the full chain.

This is standard threshold-cryptography composition (t -of- n key generation over a pre-committed quorum, then ordinary rotation); its novelty here is not cryptographic but *institutional* — the boot chain finally treats the vendor as mortal. Adversarial analysis (§7) treats the quorum as the adversary in several scenarios.

6.4 Community operation

The manifest’s accepted-operator formats matter: they pre-legitimize the actual rescue culture. If the DSM names a Home Assistant integration or an ESPHome port as an accepted community target, the successor does not start from zero, and the community’s legality improves from “gray” to “specified by the manufacturer.” The successor operator inherits obligations with the keys: vulnerability response windows, release signing rules, and telemetry defaults (death of vendor must equal death of telemetry — the manifest sets it to zero on succession, by default).

7 Threat model

Malicious vendor while alive. Sunset mode can be abused as a kill switch for market segmentation (the Sonos recycle-mode dispute is the live precedent). Mitigations: the DSM is a *public commitment* — a vendor that ships a manifest promising Tier-1 local floor and later disables it by policy has manufactured its own evidence; procurement contracts (§8) convert manifests into money; and fossil mode (§5.4) caps the worst-case physical harm regardless of vendor behavior. Honesty: nothing fully constrains a solvent vendor; the manifest’s power is reputational, contractual, and regulatory, not cryptographic.

Escrow-agent collusion. Fewer than t agents cannot release; a colluding majority can. Mitigations: agents are diversified (jurisdictions, business models — the existing escrow industry plus open archives), the release set is public on release (so the damage of a corrupt release is bounded to firmware *signing*, which the device-side ceremony still gates), and release events are on the public log, making collusion self-demonstrating.

Malicious successor (the xz problem). The most serious class: an attacker joins the community quorum and ships backdoored firmware — precisely the attack pattern of the 2024 xz-utils compromise (P-010’s subject). The succession protocol inherits P-010’s defenses: staged trust elevation for new maintainers, co-signature requirements for releases in the first window

after succession, and mandatory public review periods for any post-succession key change. A device fleet is *better positioned* than a package registry to enforce this: boot chains can enforce time-locks in hardware.

Downgrade attacks. An attacker forces early release (false trigger) to push the fleet to Tier 2 with a weaker security posture. Mitigations: sustained-silence thresholds, mirrorable renewal proofs, and the requirement that Tier-2 transitions only ever increase *local* autonomy — never disable the local floor that already exists.

Key compromise of the dead vendor’s keys. If the vendor’s private keys leak post-mortem, devices must prefer the live succession chain; the protocol’s default is that succession-root time-lock windows make vendor keys unforgeably stale after the quorum ceremony.

8 Evaluation design

A reference implementation is a test harness, not a product: a small fleet of development boards (ESP32-class) behind a fabricated “vendor” — a cloud emulator with deliberate death modes (insolvency, neglect, sunset). The evaluation answers:

1. **Survival:** fraction of capabilities preserved across each death mode, measured against the manifest’s ladder (target: Tier-1 floor 100% within 24 hours of trigger; Tier-2 community parity within 90 days for the portable set).
2. **False positives:** accidental trigger rate over a year of synthetic flapping (missed renewals, DNS failures, cert churn) — target zero with hysteresis at the declared thresholds.
3. **Ceremony soundness:** formal verification of the boot-chain acceptance rules (the threshold ceremony’s signature verification in a small TLA+-style model).
4. **Adversary replay:** re-run the xz-style malicious-successor attack against the staged elevation rules; measure whether the co-signature window blocks the backdoor publish.
5. **Fossil floor audit:** physical-interlock verification for the lock/opener/thermostat classes (the floor must survive total firmware compromise).

A second, retrospective evaluation replays real events against the model: for Insteon (insolvency), Revolv (sunset), myQ (consolidation), and Sonos (acquisition), we reconstruct which manifest tier the architecture would have held, and what the actual outcome was. The delta between the two columns is the paper’s quantified claim.

9 Economics and policy

Procurement is the first market. Hotels, property managers, and municipalities buy devices in fleets of thousands, hold them for a decade, and already read lifecycle terms (the enterprise escrow precedent). A “succession-ready” seal backed by a verifiable manifest is exactly the artifact procurement processes can require — the demand side the consumer market lacks. B2B adoption funds the tooling; consumer labels follow.

Regulation is converging on adjacent ground. The French repairability index (2021) proves longevity labeling is legislatable; the EU’s ecodesign rules already mandate support windows for some device classes; right-to-repair statutes are spreading across US states. A mandated *succession manifest* for device classes with safety-relevant fossil floors (locks, openers, thermostats) is a plausible 5-year extension of exactly these instruments — this paper supplies the specification such a rule would reference.

The dead-hand trigger as public infrastructure. Renewal-proof logs are cheap, boring, and useful to more than devices (P-010’s registry governance, P-011’s model-deprecation notices

need the same primitive). One public “liveness ledger” serves the whole continuity series — a deliberate reuse.

10 Objections

“Vendors will never opt in.” While alive and competing, most won’t. The design does not require consensus: it requires one seal, one procurement channel, and time. Enterprise escrow was also “never going to happen” until customers with leverage wrote it into contracts. The manifest’s observability converts an invisible attribute into a purchasable one; that is all a market needs to start sorting.

“You’re handing attack surface to volunteers.” The comparison is not “community firmware vs. safe status quo” — it is community firmware vs. an abandoned fleet running unmaintained cloud-coupled firmware on expired certificates, which is the actual status quo after vendor death. The successor at least patches. Staged elevation and co-signature windows (the xz defenses) address the takeover attack specifically; the fossil floor caps the physical worst case categorically.

“Bankruptcy estates own that IP.” Escrow terms are contracts executed *before* insolvency, transferring specified release rights on defined triggers; what the estate retains is what it never had in escrow. This is the exact structure forty years of software escrow practice has litigated into predictability. (Jurisdictional variation is real — see §11.)

“Matter solves this.” Matter standardizes live interoperability. It has no lifecycle semantics: no escrow, no triggers, no key succession, no floor. A Matter device whose vendor dies loses updates and cloud features exactly as a non-Matter device does. The standards body could adopt the DSM; until then, the gap is open.

“Just buy local-first devices.” Excellent practice, incomplete answer: it solves *your* next purchase, not the installed base of billions, and even local devices need firmware updates and signing authority — vendor death eventually reaches them through the same rot. Succession-readiness is complementary to local-first, not replaced by it.

11 Limitations

The protocol depends on the vendor arming it while alive; the installed base of already-sold cloud-coupled devices cannot be retrofitted, and volunteer reverse-engineering remains their only recourse. Boot chains with burned vendor-only keys are unrescuable by software; fossil floors are manufacturable only at design time. Community capacity is unevenly distributed — popular devices attract maintainers, the long tail does not; the manifest’s accepted-operator formats mitigate but do not solve coverage. Dead-hand triggers are jurisdiction- and record-sensitive (bankruptcy records differ across countries; WHOIS privacy post-GDPR erodes domain-lapse observability). And the malicious-vendor-while-alive case is capped, not solved, by manifest observability. These are real boundaries; none of them rescinds the specification, they bound its deployment.

12 Conclusion

The industry ships hardware with a decade of physical life into an ecosystem with a few years of corporate attention, and the mismatch is treated as weather. It is a specification failure with a known shape: a lifecycle state machine that pretends vendors are immortal; a capability floor that exists only in marketing copy; signing authority with no succession plan; and a rescue culture that is admired, unpaid, and technically illegal. Every component needed to fix this — escrow contracts, dead-hand triggers, threshold key ceremonies, degradation ladders, hardware interlocks, longevity labels — exists today in an adjacent industry, proven at its own scale. This paper composes them into a specification for the one transition nobody designs: the death of the maker. The devices are ready. The protocol, now, is too.

References

1. Insteon service shutdown and partial restoration (2023). Reporting: *The Verge*, April 2023 (“Insteon servers shut down, bricking hubs without warning”) and follow-ups on the community-led restoration. [Event record; to be verified against live sources in the citation-verification pass.]
2. Revolv hub shutdown by Google/Nest (2016). Reporting: *Ars Technica* / *The Verge*, May 2016, including the “Works with Nest” integration retirement. [Event record; verification queued.]
3. Chamberlain myQ local-API access blocked for Home Assistant (2023). Reporting: *The Verge*, June 2023; Home Assistant announcement of integration removal. [Event record; verification queued.]
4. Sonos legacy-product dispute and trade-in “recycle mode” (2020). Reporting: *The Verge*, January–February 2020. [Event record; verification queued.]
5. Connectivity Standards Alliance. *Matter Specification* (1.x, 2022–). Interoperability standard; no lifecycle/succession semantics. [Standards record.]
6. Gabaix, X., & Laibson, D. (2006). “Shrouded Attributes, Consumer Myopia, and Information Suppression in Competitive Markets.” *Quarterly Journal of Economics*, 121(2). [Peer-reviewed; high confidence.]
7. Source-code escrow practice. Industry standard in enterprise software contracting since the 1980s; see e.g. Iron Mountain / NCC Group escrow service descriptions. [Industry record; verification queued.]
8. French repairability index (indice de réparabilité), mandatory January 2021 (AGEC law); durability index legislated in follow-on. [Regulatory record; high confidence, details queued.]
9. U.S. FTC, “Nixing the Fix” report (2019) and 2021 policy statement against warranty-void-if-removed restrictions; state right-to-repair statutes (NY 2022, CA 2023, MA automotive 2020/2024). [Regulatory record; high confidence on FTC, details queued.]
10. EU Ecodesign Directive implementation measures mandating spare-parts availability and software-update windows for energy-related products (2019–, evolving). [Regulatory record; details queued.]
11. DMCA §1201 (17 U.S.C. §1201) anti-circumvention regime; Library of Congress triennial rulemaking exemptions for repair and security research (2018–2021 cycles). [Statutory record; high confidence.]
12. Wink hub outages and transition to paid subscription (2020). Reporting: *The Verge*, May–July 2020. [Event record; verification queued.]
13. “Works with Nest” program shutdown (2016), retiring Revolv and third-party integrations. [Event record; folded into reference 2.]

14. Borbély two-process alertness model — not cited here; see P-009 for the human-time axis of continuity. [Cross-reference.]
15. p-rick P-004, *Degradation Contracts* (this program, 2026): fidelity ladders, hysteresis, and floor invariants reused as the succession ladder’s semantics. [Program cross-reference.]
16. p-rick P-010, *The Bus Factor Protocol* (this program, 2026): staged trust elevation and co-signature windows for post-succession maintainers. [Program cross-reference.]

Working-paper note: references marked “verification queued” are recorded from domain knowledge at high confidence and await the program’s live citation-verification pass (see agents.md); the verification ledger is maintained with the paper sources.